Aran Learn 10: OCR Basics

For: Bharadhwaj and Arun (roadmap item 10). Inbathamizhan and Aasif read sections 1, 5 and 6. **Time:** about 5 hours with practice. **Before this:** Learn 1, Learn 2 (Verhoeff), and Learn 5 sections 1 and 2.

Every detector in Aran so far reads text. But a lot of identity data travels as pictures: scanned Aadhaar and PAN cards in loan files, photos of cards attached to support tickets and WhatsApp chats, PDF statements that are really images. A text scanner sees none of it. **OCR** (optical character recognition) turns those pictures into text so Aran's detectors can run on them, and tells us where on the image each word sits so we can black it out.

This doc uses only open-source tools on a laptop CPU. Every output below was produced by running the code shown. All card images are plain **specimen** cards we drew ourselves with generated test numbers; they do not copy the design of any real ID card, and nobody should ever test with a real person's card.

1. How OCR works

OCR has four stages:

1. **Clean up the image.** Fix rotation, remove noise, adjust contrast.
2. **Find the text.** Locate blocks, lines and words on the page.
3. **Recognise characters.** Turn each line's pixels into a string.
4. **Return text with positions.** For every word: the text, a box (left, top, width, height), and a confidence score.

Tesseract is the most widely used open-source OCR engine. It was started at HP, later developed by Google, and is now community-maintained. Since version 4 its recogniser is a neural network (an LSTM, a network that reads a line of pixels left to right and outputs characters), and it ships trained models for over 100 languages, including Hindi (`hin`) and Tamil (`tam`).

Other engines worth knowing: **EasyOCR** and **PaddleOCR** (deep-learning engines, often better on photos, also free), and paid cloud OCR services. Aran must run inside the bank (Learn 7: card and Aadhaar data stays in India and in the bank's systems), so sending ID card images to an outside cloud OCR is not an option. That is why this doc uses Tesseract.

2. Setup

- **Windows:** install Tesseract from the UB Mannheim installer (search "tesseract ubmannheim"), tick Hindi and Tamil under "Additional language data", and add `C:\Program Files\Tesseract-OCR` to your PATH.
- **Linux:** `sudo apt install tesseract-ocr tesseract-ocr-hin tesseract-ocr-tam`
- Then, in your virtual environment: `pip install pytesseract pillow opencv-python-headless numpy`

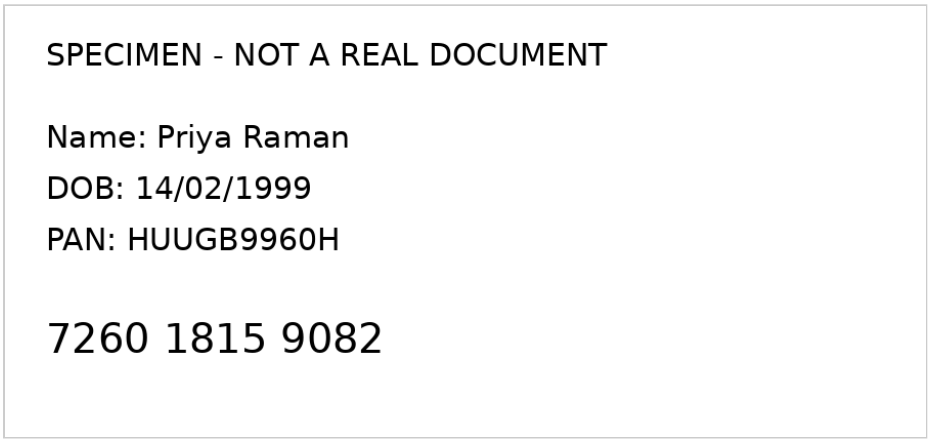
Check it with `tesseract --list-langs`. You should see `eng`, `hin` and `tam`.

`pytesseract` is a thin Python wrapper that calls the `tesseract` program for you. `pillow` (imported as `PIL`) opens and draws on images. `opencv` (imported as `cv2`) does image clean-up.

3. Your first OCR

3.1 Make a specimen card

```
# make_card.py: draw a plain SPECIMEN card with generated test values (not a copy of any real
ID design)
from PIL import Image, ImageDraw, ImageFont
F = "DejaVuSans.ttf"     # on Windows use "arial.ttf"
img = Image.new("RGB", (900, 420), "white")
d = ImageDraw.Draw(img)
big, mid = ImageFont.truetype(F, 40), ImageFont.truetype(F, 30)
d.text((40, 30), "SPECIMEN - NOT A REAL DOCUMENT", font=mid, fill="black")
d.text((40, 110), "Name: Priya Raman", font=mid, fill="black")
d.text((40, 160), "DOB: 14/02/1999", font=mid, fill="black")
d.text((40, 210), "PAN: HUUGB9960H", font=mid, fill="black")
d.text((40, 300), "7260 1815 9082", font=big, fill="black")
img.save("card.png")
```



The 12-digit number is a generated test value that passes Verhoeff (Learn 2) and belongs to no one.

3.2 Read it and check the number

```
import re, sys
import pytesseract
from PIL import Image
sys.path.insert(0, "../../research")           # path to Aran's research folder
from aran_core import verhoeff_valid

text = pytesseract.image_to_string(Image.open("card.png"))
print(text.strip())
for m in re.finditer(r"(?<!\d)(\d{4})\s?(\d{4})\s?(\d{4})(?!\d)", text):
    n = "".join(m.groups())
    print("found", n, "Verhoeff valid:", verhoeff_valid(n))
```

```
SPECIMEN - NOT A REAL DOCUMENT

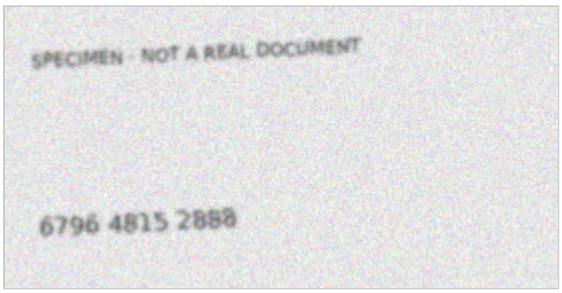
Name: Priya Raman
DOB: 14/02/1999
PAN: HUUGB9960H
```

```
7260 1815 9082
found 726018159082 Verhoeff valid: True
```

On a clean image, OCR is perfect. From here, every one of Aran's text detectors can run on the result: the PAN pattern, the name finder, the calibrated detector.

4. Real photos are messy

A phone photo of a card is small, slightly tilted, blurred and grainy. To measure what that does, we drew 200 specimen cards, each with a different generated Aadhaar number, and turned each into a fake phone photo: shrunk, tilted by up to 3 degrees, blurred, dimmed and given random noise. We did this at three sizes. Here is the smallest size:




```
def phone_photo(img, r, scale):
    """Simulate a phone photo: shrink, tilt, blur, dim, add noise."""
    w, h = img.size
    img = img.resize((int(w * scale), int(h * scale)))
    img = img.rotate(r.uniform(-3, 3), expand=True, fillcolor="white")
    img = img.filter(ImageFilter.GaussianBlur(r.uniform(0.4, 1.0)))
    a = np.asarray(img).astype(float) * 0.75 + 40
    a += np.random.default_rng(r.randrange(10**6)).normal(0, 12, a.shape)
    return Image.fromarray(np.clip(a, 0, 255).astype(np.uint8))
```

For each photo we ran OCR and sorted the outcome into four bins:

- **Right:** the exact number was read.
- **Caught:** a 12-digit number was read, but it was wrong, and it **failed Verhoeff**, so Aran knows the read is bad.
- **Wrong but valid:** a wrong number that happens to pass Verhoeff. This is the dangerous case.
- **Not found:** no 12-digit number was read at all.

Results for 200 photos at each size, with Tesseract 5.3:

Photo width (pixels)	Right	Caught by Verhoeff	Wrong but valid	Not found
552	200	0	0	0
368	196	3	0	1
276	134	21	**0**	45

Two lessons:

1. **Image size matters most.** At 552 pixels wide every number was read; at 276, a third were lost. Real systems should reject photos below a minimum size and ask the customer

to retake.

2. **Verhoeff caught every misread.** In 600 photos, not one wrong number passed the checksum. This is the same algebra Aran's tokens rest on (Learn 2): Verhoeff catches every single-digit error and every swap of two neighbouring digits, and OCR mistakes are mostly exactly that (a 3 read as 8, a 1 read as 7). So a checksum built for typing errors in 1969 doubles as an OCR quality check, for free. A failed check means "retake the photo" or "send to a person", never "accept".

4.1 Measuring OCR accuracy

When you compare engines or settings, use the **character error rate (CER)**:

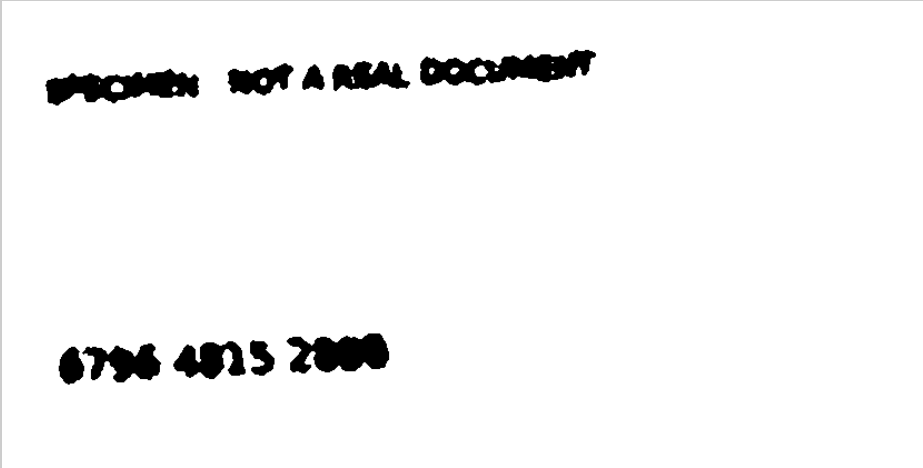
CER = (substitutions + deletions + insertions) / number of characters in the true text

Example: true `726018159082`, read `726018158082`. One substitution (9 read as 8) out of 12 characters gives CER = 1/12 = 8.3%. The `editdistance` or `jiwer` Python packages compute it for you. For Aran, also report the four bins above, because a bank cares less about average character errors than about how many wrong numbers get accepted.

5. Pre-processing: measure, don't assume

Most OCR tutorials tell you to clean the image before OCR: convert to grey, enlarge, remove speckles and turn it into pure black and white (**binarisation**). Otsu's method picks the black/white cut-off automatically from the image's brightness histogram.

```
def preprocess(img):
    g = cv2.cvtColor(np.asarray(img), cv2.COLOR_RGB2GRAY)                  # 1. grey
    g = cv2.resize(g, None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)     # 2. enlarge 3
times
    g = cv2.medianBlur(g, 3)                                               # 3. remove speckle
    _, b = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)   # 4. black and
white
    return Image.fromarray(b)
```



We ran this on the same 600 photos:

Photo width	Right, raw	Right, pre-processed	Not found, raw	Not found, pre-processed
552	200	198	0	1

Photo width	Right, raw	Right, pre-processed	Not found, raw	Not found, pre-processed
368	196	185	1	12
276	134	114	45	40

Here, pre-processing made things worse. Tesseract 5's neural recogniser already handles grey images well, and our hard black-and-white cut threw away detail it could have used. Pre-processing still helps in other cases (very uneven lighting, shadows, coloured backgrounds), so the lesson is not "never pre-process" but "test every step on your data and keep only what helps". This is the same rule as the rest of Aran: a technique counts only if it beats the baseline on identical data.

Steps that are usually worth trying, one at a time: **deskew** (rotate the text back to horizontal), **crop** to the card, **adaptive thresholding** (a different cut-off for each area of the image, for shadows), and Tesseract's **page segmentation mode** (`--psm 6` treats the image as one block of text; `--psm 7` as one line).

6. Masking on the image

Finding the number is half the job. Aran must also hide it on the image before the image is stored or shared. A **masked Aadhaar** shows only the last 4 digits, the same format UIDAI uses for its own masked Aadhaar, and the format a bank should keep outside the Aadhaar Data Vault (Learn 7, section 3.2).

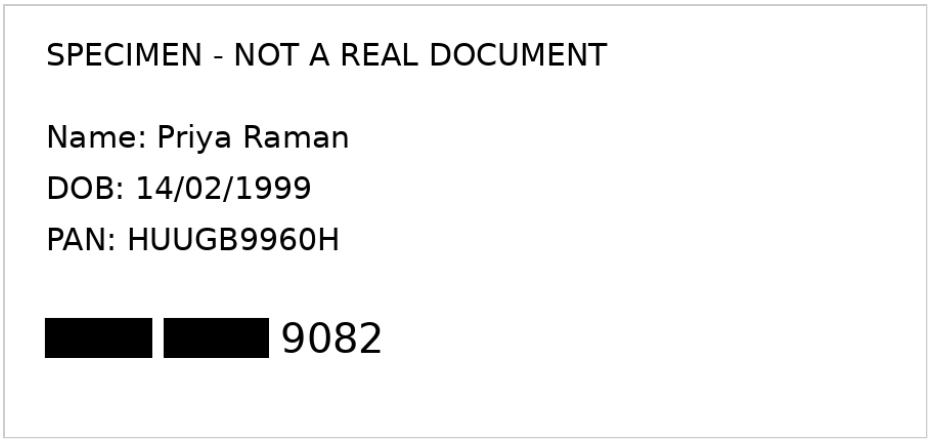
`image_to_data` returns every word with its box:

```
data = pytesseract.image_to_data(img, output_type=pytesseract.Output.DICT)
words = [(data["text"][i], data["left"][i], data["top"][i], data["width"][i], data["height"]
[i])
         for i in range(len(data["text"])) if data["text"][i].strip()]
```

```
('Name:', 43, 116, 93, 22)
('Priya', 151, 115, 69, 29)
('Raman', 235, 116, 100, 22)
('PAN:', 43, 216, 63, 22)
('HUUGB9960H', 122, 216, 203, 22)
('7260', 43, 308, 96, 30)
('1815', 159, 308, 94, 30)
('9082', 272, 308, 94, 30)
```

(Other lines trimmed.) Each tuple is the word, then the left and top position of its box in pixels, then its width and height. Find three 4-digit groups in a row that together pass Verhoeff, and paint black boxes over the first two:

```
draw = ImageDraw.Draw(img)
for i in range(len(words) - 2):                       # three 4-digit groups in a row
    group = words[i:i + 3]
    if all(re.fullmatch(r"\d{4}", g[0]) for g in group) and verhoeff_valid("".join(g[0] for g
in group)):
        for text, x, y, w, h in group[:2]:            # hide the first 8 digits
            draw.rectangle([x - 4, y - 4, x + w + 4, y + h + 4], fill="black")
img.save("masked.png")
```



Running OCR on `masked.png` now reads the number line as `Ml EE 9082`: the black boxes come out as noise and none of the first 8 digits survive. The 4-pixel margin around each box matters: a box drawn exactly on the OCR outline can leave the edges of digits visible.

Security rules for image masking:

- **Paint pixels, never add a layer.** A black rectangle drawn in a PDF viewer on top of the image can be removed, and the number is still underneath. Change the image pixels themselves and save a new file.
- **Remove metadata.** Phone photos carry EXIF data (GPS location, device). Strip it when saving.
- **The original is still raw data.** Masking makes a safe copy; the original goes to the vault or is deleted, following the same rules as any raw Aadhaar.
- **The PAN and name on the card are personal data too.** Run Aran's PAN pattern and name finder on the OCR text and mask those boxes as the sink's policy requires.
- **Hindi and Tamil text.** Real cards print the name in a regional script as well. Run OCR with `lang="eng+hin"` or `lang="eng+tam"`, and pass the text to the Indian-language name finder (Learn 9).

7. Where this fits in Aran

```
image arrives (upload, ticket, chat, PDF page)
   │
   ▼
OCR: text + word boxes
   │
   ▼
Aran detectors on the text (Verhoeff, Luhn, PAN, names)
   │
   ├─ number passes Verhoeff → mask first 8 digits on the image, vault the original
   ├─ 12 digits, fails Verhoeff → unreadable: ask to retake, or send to a person
   └─ nothing found → pass, but count it, so the immune layer sees image flows
```

The image becomes one more data flow, from the upload service to wherever the image is stored, and the firewall counts it like any other.

8. Practice

- [ ] **P1.** Install Tesseract with Hindi and Tamil, make the specimen card in 3.1, and reproduce the output in 3.2.
- [ ] **P2.** Reproduce the table in section 4 with 50 photos per size, then add a fourth size (0.2). At what width does "Not found" pass 50%?
- [ ] **P3.** Write a function that computes CER, and report the average CER per photo size for the number line only.
- [ ] **P4.** Try `--psm 7` on a crop of just the number line. Does it beat `--psm 6` on the smallest photos?
- [ ] **P5.** Add deskewing (use `cv2.minAreaRect` on the dark pixels to find the angle) and test whether it helps, using the same 200 photos with and without it.
- [ ] **P6.** Add a second line to the specimen card with a name in Tamil script (you will need a Tamil font such as Noto Sans Tamil) and read it with `lang="eng+tam"`.
- [ ] **P7.** Extend the masking code to also black out the PAN, and confirm with OCR on the masked image that neither the PAN nor the first 8 digits can be read.
- [ ] **P8.** For a misread that fails Verhoeff, list all single-digit edits (change one digit to each of the other 9 values) that pass `verhoeff_valid`. How many candidates are there on average? Explain why Aran still asks for a retake instead of guessing.

9. Further reading

- Tesseract documentation, "Improving the quality of the output" (tesseract-ocr.github.io/tessdoc/ImproveQuality.html): the official guide to pre-processing and page segmentation modes.
- Smith, "An Overview of the Tesseract OCR Engine" (ICDAR 2007), the original design paper.
- OpenCV tutorial, "Image Thresholding" (docs.opencv.org), for Otsu and adaptive thresholds.
- PaddleOCR and EasyOCR project pages on GitHub, as engines to compare against Tesseract.
- openbharatocr (PyPI), an open-source project for reading Indian ID documents, as a reference for layouts and fields.
- UIDAI, "Masked Aadhaar" (uidai.gov.in), for the masking format.